GDD — SALTARÍN

Género: Runner de un botón (auto-scroll lateral)
Plataforma: Navegador (Phaser 4 + Vite + JavaScript)
Materia: Desarrollo Tecnológico 2 · Universidad Nacional de Rafaela (UNRaf)
Profesores: Nocete, Nicolás - Galván, Agustín
Alumno: Trod, Mariano

---

1. Concepto

El jugador controla un cubo de neón que avanza solo por un escenario. Con una única tecla salta por encima de trampas. El juego tiene 3 niveles. Cada nivel agrega un tipo de obstáculo nuevo y aumenta la velocidad. Al completar los 3 niveles se gana el juego.

Frase de diseño: "Un botón, tres niveles, cero perdón."

2. Loop de juego

1. El nivel comienza y el escenario se desplaza automáticamente.
2. El jugador salta para esquivar los obstáculos.
3. Si toca una trampa, muere: se suma un intento y el nivel reinicia desde el principio.
4. Si llega al final, ve la pantalla "Nivel X completo" y pasa al siguiente.
5. Al completar el nivel 3, ve la pantalla de victoria con el total de intentos.

3. Controles

Acción	Control
Saltar / Comenzar / Continuar	Espacio o click
Reiniciar el juego (pantalla final)	R

4. Mecánicas

Mecánica principal: salto

- Altura fija. Solo se puede saltar estando apoyado (en el suelo o sobre un bloque).
- Sin doble salto. No importa cuánto tiempo se mantenga presionada la tecla.

Avance automático

- El jugador avanza en el mundo a la velocidad del nivel y la cámara lo sigue, manteniéndolo visualmente fijo en X (~25% del ancho de pantalla). Para el jugador, el escenario "viene" hacia él.
- Los obstáculos y el suelo son estáticos en el mundo (cuerpos estáticos de Arcade Physics), lo que simplifica las colisiones.

Muerte

- Tocar un pincho, caer en un pozo o chocar un bloque de costado provoca muerte instantánea.
- El nivel actual reinicia desde el principio y el contador de intentos suma 1.

5. Obstáculos

Obstáculo	Aparece desde	Comportamiento
Pincho	Nivel 1	Triángulo en el suelo (40×40 px). Hay que saltarlo. Puede venir en grupos de 1, 2 o 3 juntos.
Pozo	Nivel 2	Hueco en el suelo de ancho variable. Si el jugador cae, muere.
Bloque	Nivel 3	Rectángulo de 60 px de alto. Se puede caer y caminar encima. Chocarlo de costado mata.

6. Niveles

Los niveles se definen como **datos** (un archivo `levels.js`), no como código separado. Cada nivel tiene velocidad, largo y una lista de obstáculos con su posición en X (en píxeles desde el inicio).

Parámetros físicos iniciales (a balancear jugando)

- Gravedad: 2000 px/s²
- Velocidad de salto: −750 px/s (altura máx. ≈ 140 px, tiempo en el aire ≈ 0,75 s)
- Regla de diseño: entre el aterrizaje y el siguiente obstáculo debe haber al menos ~150 px de reacción.

Nivel 1 — "Pinchos" (aprendizaje)

Velocidad: 300 px/s · Largo: 4000 px · Distancia de salto ≈ 225 px

X	Obstáculo
600	pincho ×1
1000	pincho ×1
1400	pincho ×2
1900	pincho ×1
2150	pincho ×1 *(dos saltos seguidos: ritmo)*
2700	pincho ×3
3200	pincho ×2
3500	pincho ×1

Nivel 2 — "Pozos" (+ pozos)

Velocidad: 350 px/s · Largo: 5000 px · Distancia de salto ≈ 260 px

X	Obstáculo
600	pincho ×1
1000	pozo (ancho 120)
1500	pincho ×2
1900	pozo (ancho 160)
2400	pincho ×1
2850	pozo (ancho 100)
3300	pincho ×3
3800	pozo (ancho 140)
4150	pincho ×1
4500	pozo (ancho 160)

Nivel 3 — "Bloques" (+ bloques, todo combinado)

Velocidad: 400 px/s · Largo: 6300 px · Distancia de salto ≈ 300 px

X	Obstáculo
600	pincho ×2
1000	bloque (ancho 120)
1500	pozo (ancho 140)
1950	bloque (ancho 200) *(caminar por encima)*
2450	pincho ×1
2800	pozo (ancho 160)
3200	bloque (ancho 160)
3650	pincho ×3
4000	bloque (ancho 120)
4400	pozo (ancho 180)
4900	pincho ×2
5300	bloque (ancho 240)
5850	pincho ×1

> Las posiciones son un punto de partida. Se ajustan jugando: si un tramo resulta imposible o trivial, se modifican los datos y no el código.

7. Progreso y puntuación

- **Barra de progreso** arriba de la pantalla: muestra el % recorrido del nivel actual.

- **Intentos totales:** contador visible que suma 1 en cada muerte y no vuelve a cero entre niveles.
- **Indicador de nivel:** "Nivel 1/3".

8. Condiciones de victoria y derrota

- **Derrota (por nivel):** tocar una trampa reinicia el nivel actual.
- **Victoria:** completar el nivel 3. Se muestra "¡Ganaste!" con los intentos totales.
- **Reinicio:** desde la pantalla final, R vuelve al nivel 1 con los intentos en 0.

9. Flujo de pantallas

```
MenuScene ──(Espacio)──> GameScene (nivel N)
                          │ muere ──> reinicia GameScene (mismo nivel)
                          │ completa N<3 ──> LevelCompleteScene ──(Espacio)──> GameScene (N+1)
                          │ completa N=3 ──> VictoryScene ──(R)──> GameScene (nivel 1)
```

- **MenuScene:** título, "Espacio para jugar" y los controles.
- **LevelCompleteScene:** "Nivel X completo", intentos hasta ahora y "Espacio para continuar".
- **VictoryScene:** "¡Ganaste!", intentos totales y "R para reiniciar".

10. Estética

Estilo **geométrico neón**, dibujado completamente con formas de Phaser (sin assets externos).

Elemento	Color
Fondo	`#0d0221` (violeta casi negro)
Jugador (cuadrado 40×40)	`#00f0ff` (cian)
Pinchos	`#ff2a6d` (magenta)
Suelo	`#05d9e8` (línea/relleno cian oscuro)
Bloques	`#7b2cbf` (violeta) con borde `#d1f7ff`
Texto/UI	`#d1f7ff`

Opcional, solo si sobra tiempo: rotación del cubo durante el salto y un destello breve al morir.

11. Arquitectura y patrones

```
src/
├── scenes/     MenuScene, GameScene, LevelCompleteScene, VictoryScene
├── entities/   Player, Spike, Pit, Block
├── systems/    LevelManager (lee levels.js y spawnea), HUD (barra, intentos, nivel)
├── patterns/   ObstaclePool (Object Pool), EventBus (Observer)
├── assets/     (vacío: todo se dibuja con formas)
├── data/       levels.js
└── main.js
```

Object Pool — `ObstaclePool`

- **Problema:** en cada nivel aparecen y desaparecen decenas de obstáculos, y los reinicios son frecuentes.
- **Solución:** un pool por tipo de obstáculo. El `LevelManager` pide un obstáculo al pool cuando su X entra en pantalla por la derecha y lo devuelve al pool cuando sale por la izquierda.
- **En el reinicio:** todos los obstáculos activos vuelven al pool; no se crean objetos nuevos.

Observer — `EventBus`

- **Problema:** el HUD, el contador de intentos y el control de flujo dependen de lo que pasa en el juego, sin necesidad de conocerse entre sí.
- **Eventos:**
  - `player:died` → suma un intento y reinicia el nivel
  - `level:progress` (porcentaje) → actualiza la barra
  - `level:completed` (número de nivel) → pasa a la pantalla correspondiente
  - `attempts:changed` (total) → actualiza el HUD
- **Regla obligatoria:** toda escena o sistema que se suscriba debe desuscribirse en el evento `shutdown` de la escena. Así se evita que los listeners se dupliquen en cada reinicio.

12. Fuera de alcance

Sonido, música, más de 3 niveles, editor de niveles, guardado de récords, mobile/touch específico, selector de niveles, power-ups.